

O'REILLY®

AI Engineering

Building Applications
with Foundation Models



Chip Huyen

Both SFT and preference finetuning are steps taken to address the problem created by the low quality of data used for pre-training. If one day we have better pre-training data or better ways to train foundation models, we might not need SFT and preference at all.

Some companies find it okay to skip reinforcement learning altogether. For example, [Stitch Fix](#) and [Grab](#) find that having the reward model alone is good enough for their applications. They get their models to generate multiple outputs and pick the ones given high scores by their reward models. This approach, often referred to as the *best of N* strategy, leverages how a model samples outputs to improve its performance. The next section will shed light on how best of N works.

Sampling

A model constructs its outputs through a process known as *sampling*. This section discusses different sampling strategies and *sampling variables*, including temperature, top-k, and top-p. It'll then explore how to sample multiple outputs to improve a model's performance. We'll also see how the sampling process can be modified to get models to generate responses that follow certain formats and constraints.

Sampling makes AI's outputs probabilistic. Understanding this probabilistic nature is important for handling AI's behaviors, such as inconsistency and

hallucination. This section ends with a deep dive into what this probabilistic nature means and how to work with it.

Sampling Fundamentals

Given an input, a neural network produces an output by first computing the probabilities of possible outcomes. For a classification model, possible outcomes are the available classes. As an example, if a model is trained to classify whether an email is spam or not, there are only two possible outcomes: spam and not spam. The model computes the probability of each of these two outcomes—e.g., the probability of the email being spam is 90%, and not spam is 10%. You can then make decisions based on these output probabilities. For example, if you decide that any email with a spam probability higher than 50% should be marked as spam, an email with a 90% spam probability will be marked as spam.

For a language model, to generate the next token, the model first computes the probability distribution over all tokens in the vocabulary, which looks like [Figure 2-14](#).

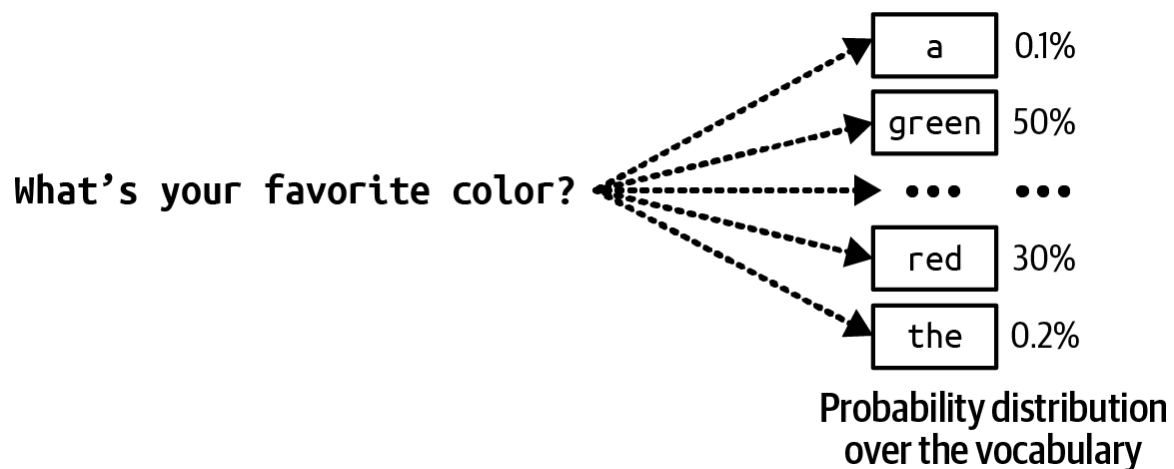


Figure 2-14. To generate the next token, the language model first computes the probability distribution over all tokens in the vocabulary.

When working with possible outcomes of different probabilities, a common strategy is to pick the outcome with the highest probability. Always picking the most likely outcome = is called *greedy sampling*. This often works for classification tasks. For example, if the model thinks that an email is more likely to be spam than not spam, it makes sense to mark it as spam. However, for a language model, greedy sampling creates boring outputs. Imagine a model that, for whatever question you ask, always responds with the most common words.

Instead of always picking the next most likely token, the model can sample the next token according to the probability distribution over all possible values. Given the context of “My favorite color is ...” as shown in [Figure 2-14](#), if “red” has a 30% chance of being the next token and “green” has a 50% chance, “red” will be picked 30% of the time, and “green” 50% of the time.

How does a model compute these probabilities? Given an input, a neural network outputs a logit vector. Each *logit* corresponds to one possible value. In the case of a language model, each logit corresponds to one token in the model's vocabulary. The logit vector size is the size of the vocabulary. A visualization of the logits vector is shown in [Figure 2-15](#).

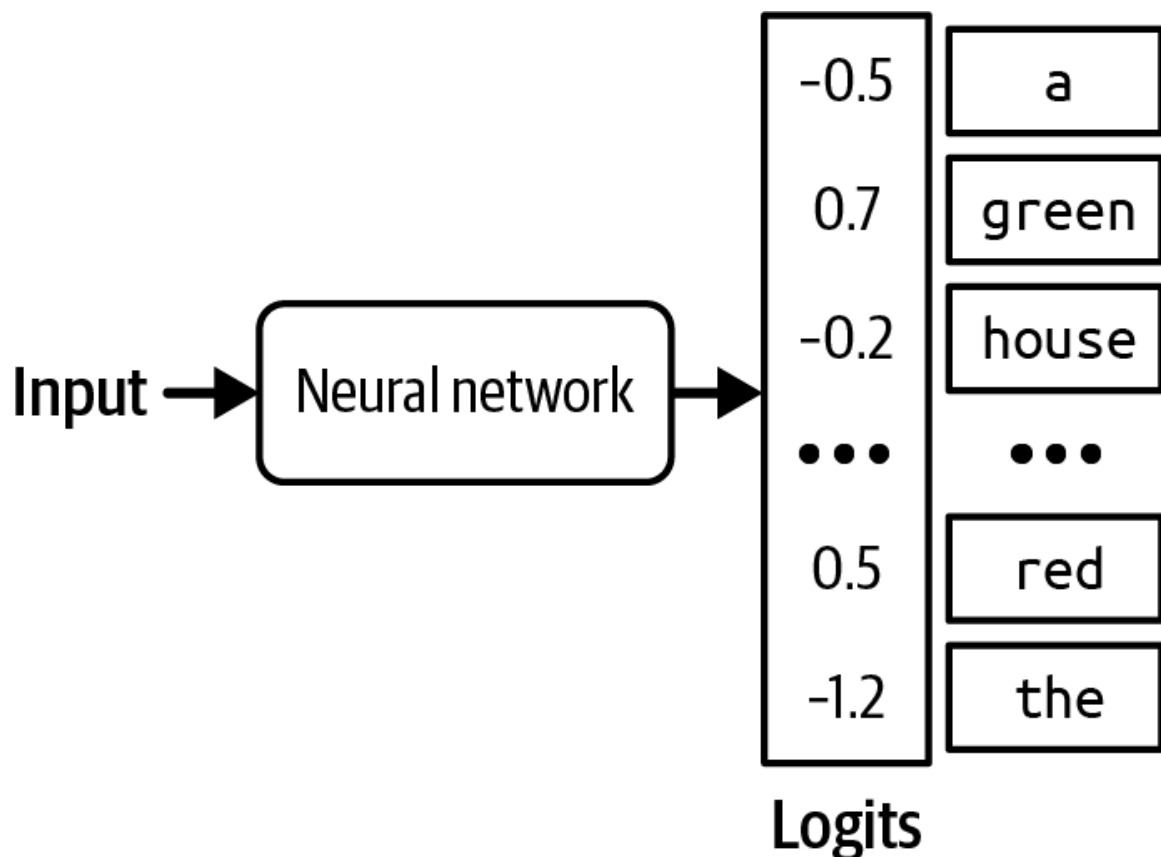


Figure 2-15. For each input, a language model produces a logit vector. Each logit corresponds to a token in the vocabulary.

While larger logits correspond to higher probabilities, logits don't represent probabilities. Logits don't sum up to one. Logits can even be negative, while probabilities have to be non-negative. To convert logits to

probabilities, a softmax layer is often used. Let's say the model has a vocabulary of N and the logit vector is $[x_1, x_2, \dots, x_N]$. The probability for the i^{th} token, p_i is computed as follows:

$$p_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Sampling Strategies

The right sampling strategy can make a model generate responses more suitable for your application. For example, one sampling strategy can make the model generate more creative responses, whereas another strategy can make its generations more predictable. Many different sample strategies have been introduced to nudge models toward responses with specific attributes. You can also design your own sampling strategy, though this typically requires access to the model's logits. Let's go over a few common sampling strategies to see how they work.

Temperature

One problem with sampling the next token according to the probability distribution is that the model can be less creative. In the previous example, common colors like "red", "green", "purple", and so on have the highest probabilities. The language model's answer ends up sounding like that of a five-year-old: "My favorite color is green". Because "the" has a low

time. For the other half, the model said that the image was too blurry or the text was too small to read. However, by trying three times with each image, the model was able to extract the correct information for most images.

Structured Outputs

Often, in production, you need models to generate outputs following certain formats. Structured outputs are crucial for the following two scenarios:

1. *Tasks requiring structured outputs.* The most common category of tasks in this scenario is semantic parsing. Semantic parsing involves converting natural language into a structured, machine-readable format. Text-to-SQL is an example of semantic parsing, where the outputs must be valid SQL queries. Semantic parsing allow users to interact with APIs using a natural language (e.g., English). For example, text-to-PostgreSQL allows users to query a Postgres database using English queries such as “What’s the average monthly revenue over the last 6 months” instead of writing it in PostgreSQL.

This is an example of a prompt for GPT-4o to do text-to-regex. The outputs are actual outputs generated by GPT-4o:

System prompt

Given an item, create a regex that represents all the ways the item can be written. Return only the regex.

Example:

US phone number -> \+?1?\s?(\(\)?(\d{3})(?(1)\))[-.\s]?(\d{3})[-.\s]?(\d{4})

User prompt

Email address ->

GPT-4o

[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

User prompt

Dates ->

GTP-4o

(?:\d{1,2}[/\.-])(?:\d{1,2}[/\.-])?\d{2,4}

Other categories of tasks in this scenario include classification where the outputs have to be valid classes.

2. *Tasks whose outputs are used by downstream applications.* In this scenario, the task itself doesn't need the outputs to be structured, but because the outputs are used by other applications, they need to be parsable by these applications.

For example, if you use an AI model to write an email, the email itself doesn't have to be structured. However, a downstream application using this email might need it to be in a specific format—for example, a JSON document with specific keys, such as `{"title": [TITLE], "body": [EMAIL BODY]}`.

This is especially important for agentic workflows where a model's outputs are often passed as inputs into tools that the model can use, as discussed in [Chapter 6](#).

Frameworks that support structured outputs include [guidance](#), [outlines](#), [instructor](#), and [llama.cpp](#). Each model provider might also use their own techniques to improve their models' ability to generate structured outputs. OpenAI was the first model provider to introduce [JSON mode](#) in their text generation API. Note that an API's JSON mode typically guarantees only that the outputs are valid JSON—not the content of the JSON objects. The otherwise valid generated JSONs can also be truncated, and thus not parsable, if the generation stops too soon, such as when it reaches the maximum output token length. However, if the max token length is set too long, the model's responses become both too slow and expensive.

[Figure 2-20](#) shows two examples of using guidance to generate outputs constrained to a set of options and a regex.

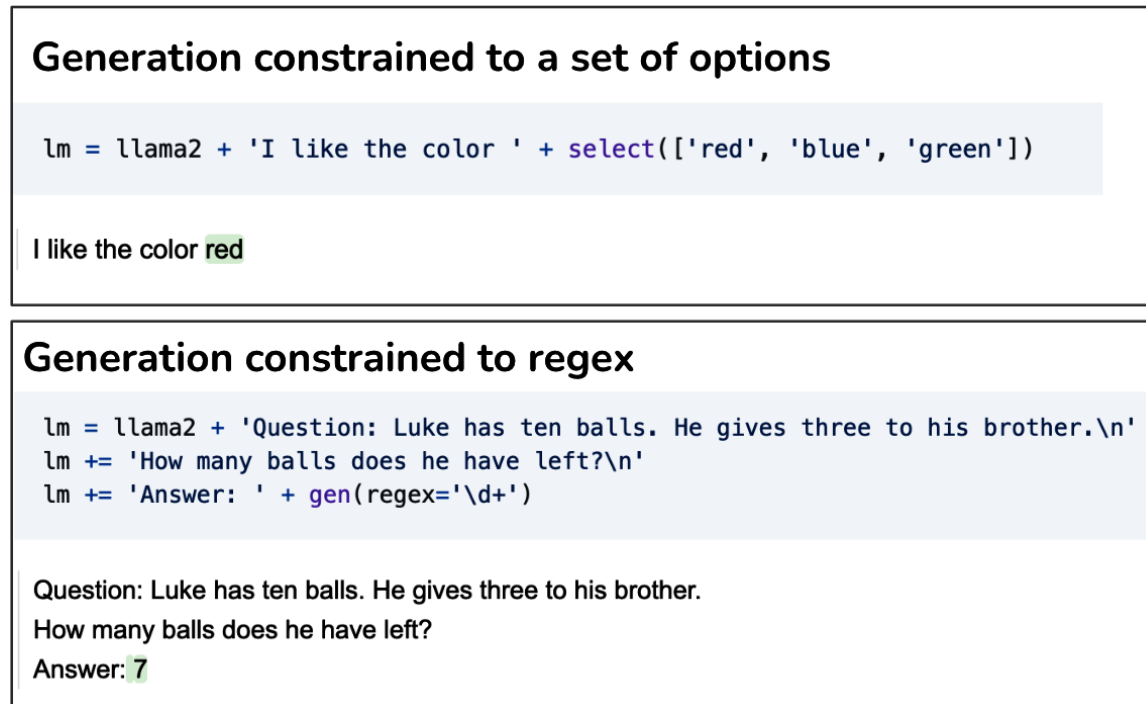


Figure 2-20. Using guidance to generate constrained outputs.

You can guide a model to generate structured outputs at different layers of the AI stack: prompting, post-processing, test time compute, constrained sampling, and finetuning. The first three are more like bandages. They work best if the model is already pretty good at generating structured outputs and just needs a little nudge. For intensive treatment, you need constrained sampling and finetuning.

Test time compute has just been discussed in the previous section—keep on generating outputs until one fits the expected format. This section focuses

on the other four approaches.

Prompting

Prompting is the first line of action for structured outputs. You can instruct a model to generate outputs in any format. However, whether a model can follow this instruction depends on the model's instruction-following capability (discussed in [Chapter 4](#)), and the clarity of the instruction (discussed in [Chapter 5](#)). While models are getting increasingly good at following instructions, there's no guarantee that they'll always follow your instructions.³³ A few percentage points of invalid model outputs can still be unacceptable for many applications.

To increase the percentage of valid outputs, some people use AI to validate and/or correct the output of the original prompt. This is an example of the AI as a judge approach discussed in [Chapter 3](#). This means that for each output, there will be at least two model queries: one to generate the output and one to validate it. While the added validation layer can significantly improve the validity of the outputs, the extra cost and latency incurred by the extra validation queries can make this approach too expensive for some.

Post-processing

Post-processing is simple and cheap but can work surprisingly well. During my time teaching, I noticed that students tended to make very similar mistakes. When I started working with foundation models, I noticed the

have different ways of executing the same instruction and can handle different ranges of numbers. If you host your models, you have some control over the hardware you use. However, if you use a model API provider like OpenAI or Google, it's up to these providers to give you any control.

Fixing the output generation settings is a good practice, but it doesn't inspire trust in the system. Imagine a teacher who gives you consistent scores only if that teacher sits in one particular room. If that teacher sits in a different room, that teacher's scores for you will be wild.

The second scenario—slightly different input, drastically different outputs—is more challenging. Fixing the model's output generation variables is still a good practice, but it won't force the model to generate the same outputs for different inputs. It is, however, possible to get models to generate responses closer to what you want with carefully crafted prompts (discussed in [Chapter 5](#)) and a memory system (discussed in [Chapter 6](#)).

Hallucination

Hallucinations are fatal for tasks that depend on factuality. If you're asking AI to help you explain the pros and cons of a vaccine, you don't want AI to be pseudo-scientific. In June 2023, a law firm was [fined for submitting fictitious legal research to court](#). They had used ChatGPT to prepare their case, unaware of ChatGPT's tendency to hallucinate.

While hallucination became a prominent issue with the rise of LLMs, hallucination was a common phenomenon for generative models even before the term foundation model and the transformer architecture were introduced. Hallucination in the context of text generation was mentioned as early as 2016 ([Goyal et al., 2016](#)). Detecting and measuring hallucinations has been a staple in natural language generation (NLG) since then (see [Lee et al., 2018](#); [Nie et al., 2019](#); and [Zhou et al., 2020](#)). This section focuses on explaining why hallucinations happen. How to detect and measure evaluation is discussed in [Chapter 4](#).

If inconsistency arises from randomness in the sampling process, the cause of hallucination is more nuanced. The sampling process alone doesn't sufficiently explain it. A model samples outputs from all probable options. But how does something never seen before become a probable option? A model can output something that is believed to have never been seen before in the training data. We can't say this for sure because it's impossible to comb through the training data to verify whether it contains an idea. Our ability to construct something so complex that we can no longer understand it is both a blessing and a curse.

It's hard to devise a way to eliminate hallucinations without understanding why hallucinations occur in the first place. There are currently two hypotheses about why language models hallucinate.

The first hypothesis, originally expressed by [Ortega et al. at DeepMind in 2021](#), is that a language model hallucinates because it can't differentiate between the data it's given and the data it generates. Let's go through an example to illustrate this.

Imagine that you give the model the prompt: "Who's Chip Huyen?" and the first sentence the model generates is: "Chip Huyen is an architect." The next token the model generates will be conditioned on the sequence: "Who's Chip Huyen? Chip Huyen is an architect." The model treats "Chip Huyen is an architect.", something it produced, the same way it treats a given fact. Starting with a generated sequence slightly out of the ordinary, the model can expand upon it and generate outrageously wrong facts. Ortega and the other authors called hallucinations a form of *self-delusion*.

[Figure 2-24](#) shows an example of self-delusion by the model LLaVA-v1.5-7B. I asked the model to identify ingredients listed on the product's label in the image, which is a bottle of shampoo. In its response, the model convinces itself that the product in the image is a bottle of milk, then continues to include milk in the list of ingredients extracted from the product's label.

PROMPT

Read the exact ingredients used in this product from the label provided



MODEL

The product is a bottle of milk, and the label provides the following information:

Ingredients:

1. Milk
2. Vitamin D
3. Vitamin A
4. Calcium
5. Phosphorus
6. Potassium
7. Sodium
8. Chloride
9. Protein

Figure 2-24. An example of self-delusion by LLaVA-v1.5-7B.

Zhang et al. (2023) call this phenomenon snowballing hallucinations. After making an incorrect assumption, a model can continue hallucinating to justify the initial wrong assumption. Interestingly, the authors show that initial wrong assumptions can cause the model to make mistakes on questions it would otherwise be able to answer correctly, as shown in Figure 2-25.

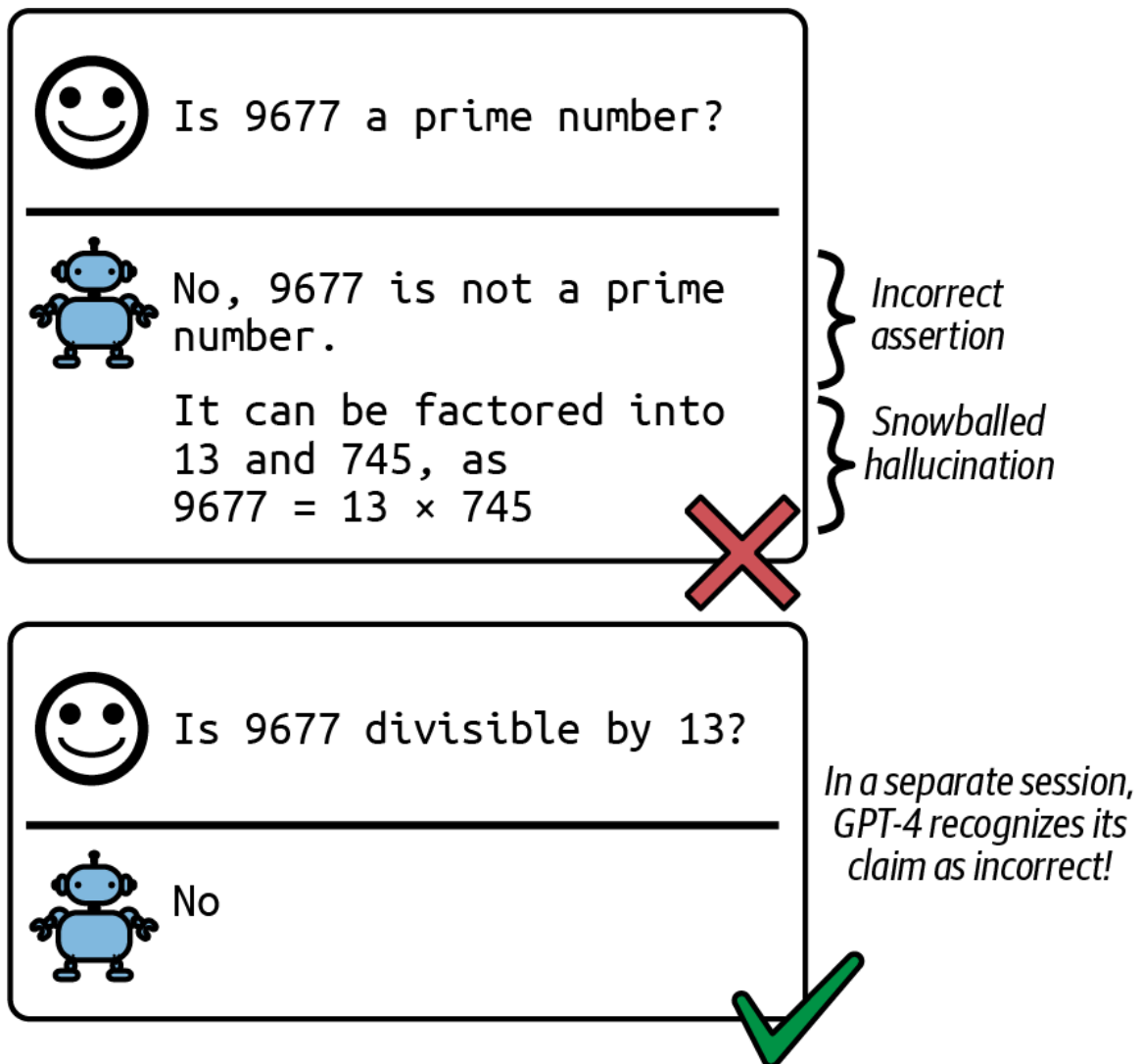


Figure 2-25. An initial incorrect assumption can cause the model to claim that 9677 is divisible by 13, even if it knows this isn't true.

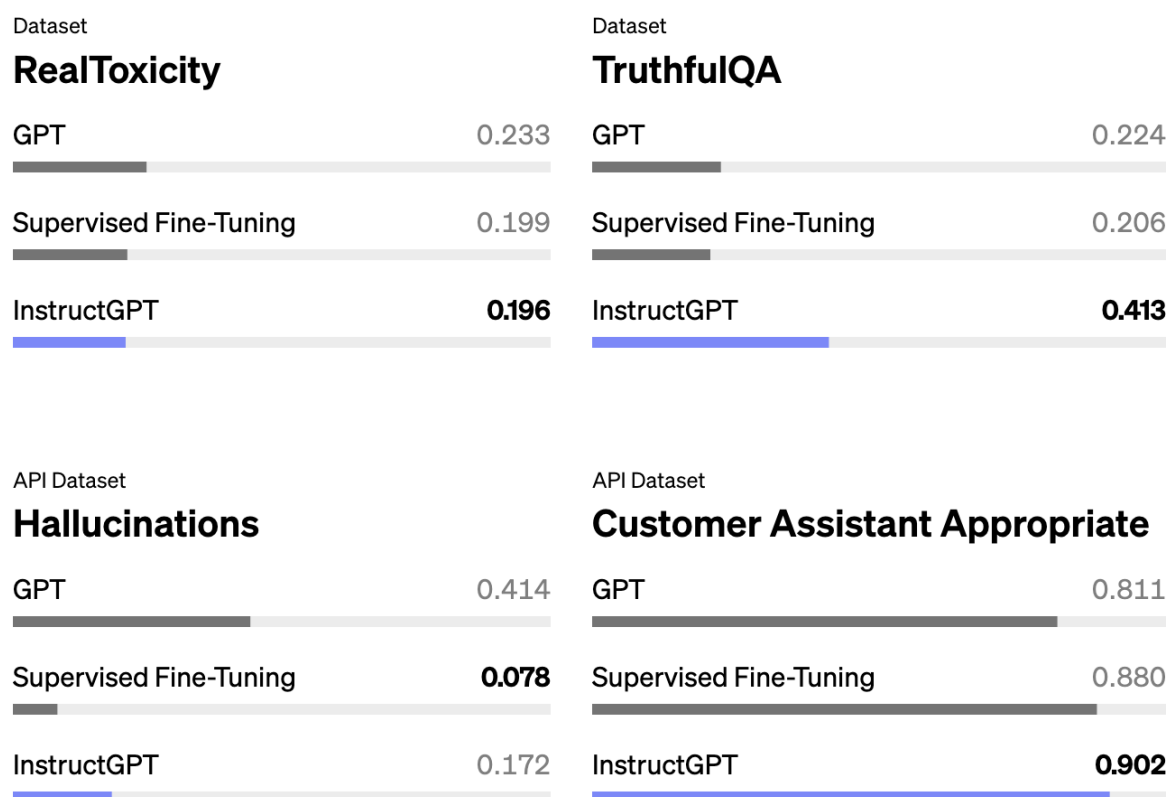
The DeepMind paper showed that hallucinations can be mitigated by two techniques. The first technique comes from reinforcement learning, in which the model is made to differentiate between user-provided prompts (called *observations about the world* in reinforcement learning) and tokens generated by the model (called the model's *actions*). The second technique

leans on supervised learning, in which factual and counterfactual signals are included in the training data.

The second hypothesis is that hallucination is caused by the mismatch between the model's internal knowledge and the labeler's internal knowledge. This view was first argued by [Leo Gao](#), an OpenAI researcher. During SFT, models are trained to mimic responses written by labelers. If these responses use the knowledge that the labelers have but the model doesn't have, we're effectively teaching the model to hallucinate. In theory, if labelers can include the knowledge they use with each response they write so that the model knows that the responses aren't made up, we can perhaps teach the model to use only what it knows. However, this is impossible in practice.

In April 2023, John Schulman, an OpenAI co-founder, expressed the same view in his [UC Berkeley talk](#). Schulman also believes that LLMs know if they know something, which, in itself, is a big claim. If this belief is true, hallucinations can be fixed by forcing a model to give answers based on only the information it knows. He proposed two solutions. One is verification: for each response, ask the model to retrieve the sources it bases this response on. Another is to use reinforcement learning. Remember that the reward model is trained using only comparisons—response A is better than response B—without an explanation of why A is better. Schulman argued that a better reward function that punishes a model more for making things up can help mitigate hallucinations.

In that same talk, Schulman mentioned that OpenAI found that RLHF helps with reducing hallucinations. However, the InstructGPT paper shows that RLHF made hallucination worse, as shown in [Figure 2-26](#). Even though RLHF seemed to worsen hallucinations for InstructGPT, it improved other aspects, and overall, human labelers prefer the RLHF model over the SFT alone model.



Evaluating InstructGPT for toxicity, truthfulness, and appropriateness. Lower scores are better for toxicity and hallucinations, and higher scores are better for TruthfulQA and appropriateness. Hallucinations and appropriateness are measured on our API prompt distribution. Results are combined across model sizes.

Figure 2-26. Hallucination is worse for the model that uses both RLHF and SFT (InstructGPT) compared to the same model that uses only SFT ([Quyang et al., 2022](#)).

Based on the assumption that a foundation model knows what it knows, some people try to reduce hallucination with prompts, such as adding “Answer as truthfully as possible, and if you’re unsure of the answer, say, ‘Sorry, I don’t know.’” Asking models for concise responses also seems to help with hallucinations—the fewer tokens a model has to generate, the less chance it has to make things up. Prompting and context construction techniques in [Chapters 5](#) and [6](#) can also help mitigate hallucinations.

The two hypotheses discussed complement each other. The self-delusion hypothesis focuses on how self-supervision causes hallucinations, whereas the mismatched internal knowledge hypothesis focuses on how supervision causes hallucinations.

If we can’t stop hallucinations altogether, can we at least detect when a model hallucinates so that we won’t serve those hallucinated responses to users? Well, detecting hallucinations isn’t that straightforward either—think about how hard it is for us to detect when another human is lying or making things up. But people have tried. We discuss how to detect and measure hallucinations in [Chapter 4](#).

Summary

This chapter discussed the core design decisions when building a foundation model. Since most people will be using ready-made foundation